CAF — Quality, Risk & Generation

Aggregated export — 2026-07-10. Source markdown in repo is unchanged.

Bundle id: `05-caf-quality-risk-generation`

Included: docs/QUALITY_CHECKS.md

Quality checks (QC)

CAF **QC** is the automated pass over **LLM output** using **checklist rows**, **copy-quality patterns**, `risk_policies`, and `brand_constraints.banned_words`. It runs in `src/services/qc-runtime.ts` after generation. **Project `risk_rules` are not enforced** — see `GET /v1/projects/:slug/risk-qc-status` and `docs/RISK_RULES.md`.

What "QC" means here

- **Not** a separate product — it is `runQcForJob(db, jobId, requireHumanReviewAfterQc)`.
- **Input:** Parsed `generation_payload.generated_output` (JSON object). Carousel flows may be **normalized** before checks (`normalizeLlmParsedForSchemaValidation`).
- **Output:** Updates `content_jobs`: `qc_status`, `generation_payload.qc_result`, `recommended_route` (column), `updated_at`.

Data model

Table	Role
`caf_core.flow_definitions`	Per `flow_type`: `qc_checklist_name`, `qc_checklist_version` point to the checklist set.
`caf_core.qc_checklists`	Rows keyed by `(qc_checklist_name, qc_checklist_version, check_id)`: `check_type`, `field_path`, `operator`, `threshold_value`, `blocking`, `severity`, etc.

Loaded via `getFlowDefinition` + `listQcChecks` in `src/repositories/flow-engine.ts`.

Check types (implemented)

The runtime `runCheck` switch in `qc-runtime.ts` supports types such as:

- `required_keys` — semicolon-separated `field_path` values must resolve.
- `equals` — includes **carousel slide-count** logic when the check looks like a slide-count rule (see `tryCarouselSlideCountEquals`).
- `min_length` / `max_length` — string or array length vs threshold.
- `regex` — string field vs pattern in `threshold_value`.
- `not_empty` — value present and non-trivial.

Unknown `check_type` defaults to **pass** (non-blocking).

Pass / fail semantics

- `qc_passed` is false if any **blocking** check fails **or** any risk finding is **CRITICAL** (risk is evaluated in the same function — see `RISK_RULES.md` for policies).
- `qc_score` = fraction of checklist rows that passed (1 if no checks).

Recommended route after QC

Derived from checklist failures + risk findings + `CAF_REQUIRE_HUMAN_REVIEW_AFTER_QC` (default **true**): even a clean checklist may force `HUMAN_REVIEW` instead of `AUTO_PUBLISH`.

Special routes: `BLOCKED`, `DISCARD`, `REWORK_REQUIRED`, `HUMAN_REVIEW`.

Payload shape

`generation_payload.qc_result` is built by `buildQcResultPayload` — includes `passed`, `score`, `recommended_route`, `reason_short`, `reasons`, optional `blocking_checks` / `blocking_risk_policies`.

Downstream UIs read this for tooltips and filters.

Typed subset (Zod) + canonical write path

`src/domain/generation-payload-qc.ts` carves out the `qc_result` slice of `generation_payload`:

- `qcResultSchema` — Zod schema describing the persisted shape.
- `mergeGenerationPayloadQc(db, jobId, qc, { qc_status, recommended_route })` — the single sanctioned writer. Validates with

`qcResultSchema.parse`, merges `qc_result` into `generation_payload`, and updates `qc_status` + `recommended_route` in one SQL statement.
- **`pickStoredQcResult`** — tolerant reader that also accepts pre-migration rows.

`runQcForJob` calls `mergeGenerationPayloadQc`; any other path that writes `qc_result` is considered drift.

Operational notes

- If **`flow_definitions`** has no **`qc_checklist_name`**, **`listQcChecks`** returns **empty** → **qc_score = 1** and only **risk policies + brand bans** apply.
- **Output schema validation** (Flow Engine **`output_schemas`**) is a **separate** step in **`llm-generator.ts`**. Rollout is controlled by **`CAF_OUTPUT_SCHEMA_VALIDATION_MODE`** (**`skip`** / **`warn`** / **`enforce`**); legacy **`CAF_SKIP_OUTPUT_SCHEMA_VALIDATION`** still works as the fallback. Not the same as QC checklist rows.

Related docs

- RISK_RULES.md — keyword policies merged into the same `runQcForJob` pass
- layers/job-pipeline.md — when QC runs in the pipeline
- GENERATION_GUIDANCE.md — separate from QC (prompt injection)

Included: docs/RISK_RULES.md

Risk rules and policies

CAF uses **three different concepts** that sound similar. This doc separates them as implemented today.

1. `caf_core.risk_policies` (QC runtime — used)

Table: CAF-wide catalog (**`risk_policy_name`**, **`risk_policy_version`**).
Loaded by: **`listRiskPolicies(db)`** in **`src/repositories/flow-engine.ts`** — currently **`SELECT * FROM caf_core.risk_policies ORDER BY risk_policy_name`** (no filter by flow or project).

Applied in: **`src/services/qc-runtime.ts`** → **`runRiskPolicy`**, inside **`runQcForJob`**.

Mechanism:

- `detection_method` — typically `keyword` or `both`.
- `detection_terms` — semicolon-separated terms; matched against **lower-cased JSON.stringify of** `generated_output` using `riskDetectionTermMatches` (word boundaries for single words).
- **Brand overlap:** `brand_constraints.banned_words` (per project) are **also** passed as extra terms for the same scan.

Effects: Builds `risk_findings`; contributes to `risk_level` (**CRITICAL** / **HIGH** / **MEDIUM** / **LOW**); can set `recommended_route` to `BLOCKED`, `DISCARD`, `REWORK_REQUIRED`, or push `HUMAN_REVIEW` based on severity and `block_publish`.

Scope: As of migration `024_risk_policies_scope.sql`, `risk_policies` has an optional `applies_to_flow_type`. When set, the policy only runs for jobs with that `flow_type`; when `NULL`, the policy is global. `runQcForJob` calls `listRiskPoliciesForJob(db, job.flow_type)` which returns the union (global + flow-scoped). Pre-existing rows default to `NULL` and keep their previous behavior.

---

2. `caf_core.risk_rules` (project config — not used by `qc-runtime`)

Table: Per-`project_id` and `flow_type` rows (migration `002_project_config_and_runs.sql`): trigger conditions, risk level, manual review flags, sensitive topics, etc.

Used for: Project profile APIs, **CSV import/export** (`src/routes/project-config.ts`, `project-csv-import.ts`), admin/bootstrap counts.

Not used for: Automated keyword QC in `runQcForJob`. Grep `qc-runtime.ts` — there is **no** query to `risk_rules`.

Implication: Operators may fill **project risk rules** expecting automated enforcement; **today** enforcement comes from `risk_policies` + **brand** `banned_words`, not from this table.

Surfacing in API responses: `GET/POST/DELETE /v1/projects/:project_slug/project-risk-rules` (preferred) and the deprecated aliases `.../risk-rules` both attach a stable `risk_qc` notice from `riskRulesNotEnforcedNotice()` (`src/services/risk-qc-status.ts`). The CSV import (`POST /v1/projects/import-csv`, `importProjectFromCsv`) also

appends an entry to `warnings[]` whenever risk_rule rows are applied. UIs should render these prominently.

Honesty endpoint: `GET /v1/projects/:project_slug/risk-qc-status` returns the QC sources and a count of the project's configured **project risk rules** (`caf_core.risk_rules`). It is sourced from `src/services/risk-qc-status.ts` → `buildRiskQcStatus` . Shape:

```
{
  "ok": true,
  "project_slug": "...",
  "qc_uses": ["risk_policies", "brand_banned_words"],
  "project_risk_rules_count": 0,
  "risk_rules_enforced_by_qc": false,
  "has_unenforced_risk_rules": false,
  "message": "...",
  "docs_path": "docs/RISK_RULES.md"
}
```

---

3. Brand constraints (`banned_words`)

Table: `caf_core.brand_constraints` (per project).
Field: `banned_words` — typically semicolon-separated.

QC: Split and merged into the **same keyword scan** as `risk_policies` (`runRiskPolicy`).

Summary table

Source	Scope	Used in automated QC?
`risk_policies`	Global CAF catalog	**Yes** — keyword scan on output JSON
`risk_rules` (project risk rules)	Per project + flow	**No** (in current `qc-runtime`)
`brand_constraints.banned_words`	Per project	**Yes** — merged into policy scan

Route changes vs QC

`recommended_route` on `content_jobs` is updated during QC. `src/decision_engine/route_selector.ts` chooses routes at **planning** time from candidate data — a different phase.

Related docs

- QUALITY_CHECKS.md — checklist + risk in one `runQcForJob` pass
- GENERATION_GUIDANCE.md — not risk enforcement; prompt steering
- ARCHITECTURE.md

Included: docs/GENERATION_GUIDANCE.md

Generation guidance

Generation guidance is **text** (and related metadata) merged into **LLM prompts** so the model steers toward brand, strategy, or operator intent. It is **not** the same as **QC** (post-hoc validation) or **plan-time learning rules** (scoring only).

Primary mechanism: `getLearningContextForGeneration`

Facade: `src/services/learning-rule-selection.ts` — `getLearningContextForGeneration(db, projectId, flow, platform, opts)`.
Implementation: `src/services/learning-context-compiler.ts` — `compileLearningContexts`.
Called from: `src/services/llm-generator.ts` during `generateForJob`, and `src/routes/learning.ts` for the context preview endpoint. Always reach the compiler through the facade — direct imports of `compileLearningContexts` in new code are considered drift.

Which rules are included

1. Load rules for the project scope (**project-scoped only**; global learning is currently disabled).
2. **Active** rules where `status === 'active'` and the row is classified as a **generation** rule:
   - `rule_family === 'generation'`, or
   - `action_type` matches `/GENERATION|GUIDANCE|HINT/i`.

3. Optional **pending** guidance on **editorial rework** only: if `include_pending_generation_guidance` is true, `pending` rules with generation-guidance-style `action_type` are also merged (same file).

Scoping

`matchesScope` filters by:

- `scope_flow_type` — exact or `*` wildcard pattern vs job `flow_type`.
- `scope_platform` — must match job platform when set.

Text extraction

Guidance text is taken from `action_payload`: first non-empty of `guidance`, `hint`, `text`, `message`, `summary`.

Injection into the prompt

`llm-generator.ts`:

1. Places `global_learning_context`, `project_learning_context`, `learning_guidance` into the **template context** (with **char caps** from `LLM_LEARNING_*` env in `config.ts`).
2. Appends **merged guidance** to the **system prompt** under a fixed preamble: *"Validated learning context … do not quote verbatim"*.

So guidance is **advisory** — the model may still ignore it.

Anti-repetition (carousel)

Separate from `learning_rules`: `buildLlmApprovalAntiRepetitionBlock` adds a block from **recent approved** jobs' copy (config `LLM_APPROVAL_ANTI_REPETITION_*`). Also appended to **system** for carousel flows when enabled.

Editorial rework overrides

When `generation_reason` / `rework_mode` indicate rework, **reviewer notes** and `editorial_overrides_json` fields are merged into the **user** prompt (not the learning compiler). See `llm-generator.ts` block around `isEditorialRework`.

Planning-time learning (different path)

The planning path is also fronted by the facade: `getLearningRulesForPlanning(db, projectId)` (`src/services/learning-rule-selection.ts`) wraps

`listActiveAppliedLearningRules` (`src/repositories/core.ts`) and returns only **ranking-style** rules (`BOOST_RANK` , `SCORE_BOOST` , `SCORE_PENALTY`) for `decideGenerationPlan` . Those **do not** inject prompt text — they change **which jobs get planned**. See `decision_engine/ranking_rules.ts` .

Attribution

`caf_core.learning_generation_attribution` records applied rule ids and context sizes (`src/repositories/learning-evidence.ts` , called from generation path). `job_outcomes` links publish to performance metrics. Run context snapshots via `setRunContextSnapshot()` capture prompt versions + learning fingerprints at plan time (failures logged, never abort run).

Related docs

- QUALITY_CHECKS.md
- RISK_RULES.md
- layers/learning.md
- layers/generation.md

Included: docs/stage-contracts/validation-output.md

Validation output (human review) — contract v1

This document defines the **stored output of the validation layer** (human review) as persisted in:

- `caf_core.editorial_reviews.validation_output_json`
- mirrored (latest only) into `caf_core.content_jobs.review_snapshot.validation_output`

The purpose is to make review decisions **automation-friendly** and **learning-ready**:

- Verdict + who/when
- Reviewed content (finalized/edited fields)
- Standardized labels (issue tags)
- Rework hints (routing controls, provider overrides)
- Findings (structured “what’s wrong and where”, with actionable guidance)

Top-level shape

`ValidationOutputV1` :

```
{
  "schema_version": "v1",
  "submitted_at": "2026-04-30T12:34:56.789Z",
  "decision": "NEEDS_EDIT",
  "validator": "migue",
  "notes": "Optional notes for downstream",
  "issue_tags": ["tone_off", "hook_strategy_wrong"],
  "content_kind": "carousel",
  "reviewed_content": {
    "title": "Final title (optional)",
    "hook": "Final hook (optional)",
    "caption": "Final caption (optional)",
    "hashtags": "#a #b (optional)",
    "slides": [
      { "index": 0, "headline": "Slide 1 title", "body": "Slide 1 body"
}
    ],
    "spoken_script": "Video-only (optional)"
  },
  "rework_hints": {
    "regenerate": false,
    "rewrite_copy": true,
    "skip_video_regeneration": false,
    "skip_image_regeneration": false,
    "heygen_avatar_id": "optional",
    "heygen_voice_id": "optional",
    "heygen_force_rerender": false
  },
  "findings": [
    {
      "label": "bad_structure",
      "severity": "warn",
      "location": { "area": "slide_body", "slide_index": 2 },
      "message": "Slide 3 body is too long and loses the core point.",
      "suggestion": "Reduce to one sentence + a single proof point.",
      "example_fix": "Replace body with: 'Here’s the one lever that
matters…'"
    }
  ],
  "metadata": {}
}
```

Notes

- `issue_tags` : today these are the review console “Issue tags” buttons; they are standardized labels.
- `reviewed_content` : this stores the *reviewed* fields (overrides when present, otherwise generated output).
- `rework_hints.regenerate` : if `false` , downstream should **reuse the existing rendered assets** and only patch copy fields (caption/hashtags/hook/etc). No renderer/HeyGen calls.
- `findings` : this is where we’ll evolve to store “what’s wrong and where” with concrete suggestions. It’s currently empty unless a caller populates it (UI work to follow).